Greedy Algorithm: Dijkstra's Algorithm

Design and Analysis of Algorithms

Dhannya SM

Dept of CSE
SSN College of Engineering

Problem Statement

- Dijkstra's Algorithm solves the single-source shortest-paths problem: for a given vertex called the source in a weighted connected graph, find shortest paths to all its other vertices.
- This is different from finding the single shortest path that starts at the source and visits all the other vertices.
- Most widely used applications are transportation planning and packet routing in communication networks
- Lesser known applications include finding shortest paths in social networks, speech recognition, document formatting, robotics, compilers, and airline crew scheduling.

Algorithm idea

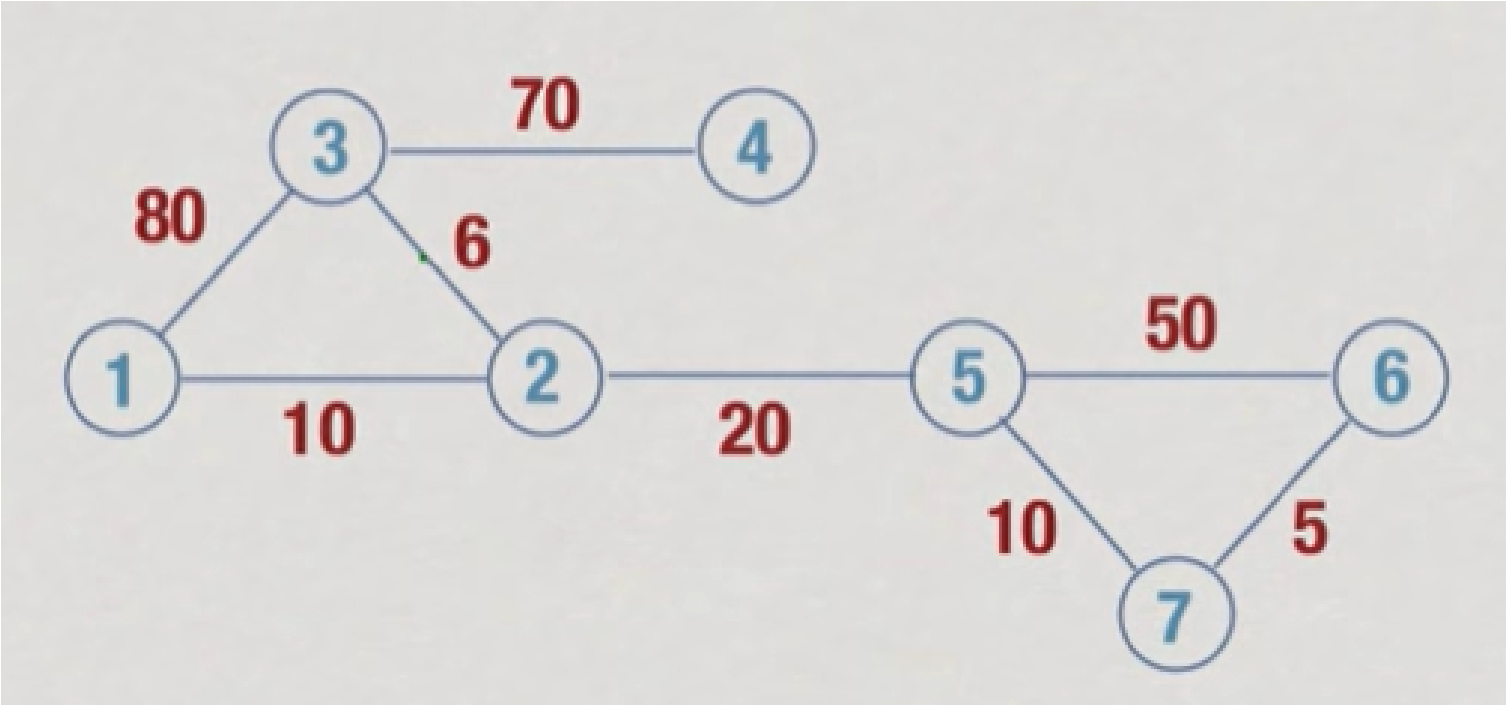
- Dijkstra's Algorithm is applicable to undirected and directed graphs with non-negative weights only.
- Dijkstra's algorithm finds the shortest paths to a graph's vertices in order of their distance from a given source.
- First, it finds the shortest path from the source to a vertex nearest to it, then to a second nearest, and so on.
- In general, before its ith iteration commences, the algorithm has already identified the shortest paths to $i-1$ other vertices nearest to the source.
- These vertices, the source vertex, and the edges of the shortest paths leading to them from the source form a subtree T_i of the given graph.

Algorithm idea

- The approach used here is called **greedy**.
- Computer scientists consider it a general design technique despite the fact that it is applicable to optimization problems only.
- The greedy approach suggests constructing a solution through a sequence of steps, each expanding a partially constructed solution obtained so far, until a complete solution to the problem is reached.
- On each step, the choice made must be:
  - *feasible*, i.e., it has to satisfy the problem's constraints
  - *locally optimal*, i.e., it has to be the best local choice among all feasible choices available on that step
  - *irrevocable*, i.e., once made, it cannot be changed on subsequent steps of the algorithm

Algorithm idea

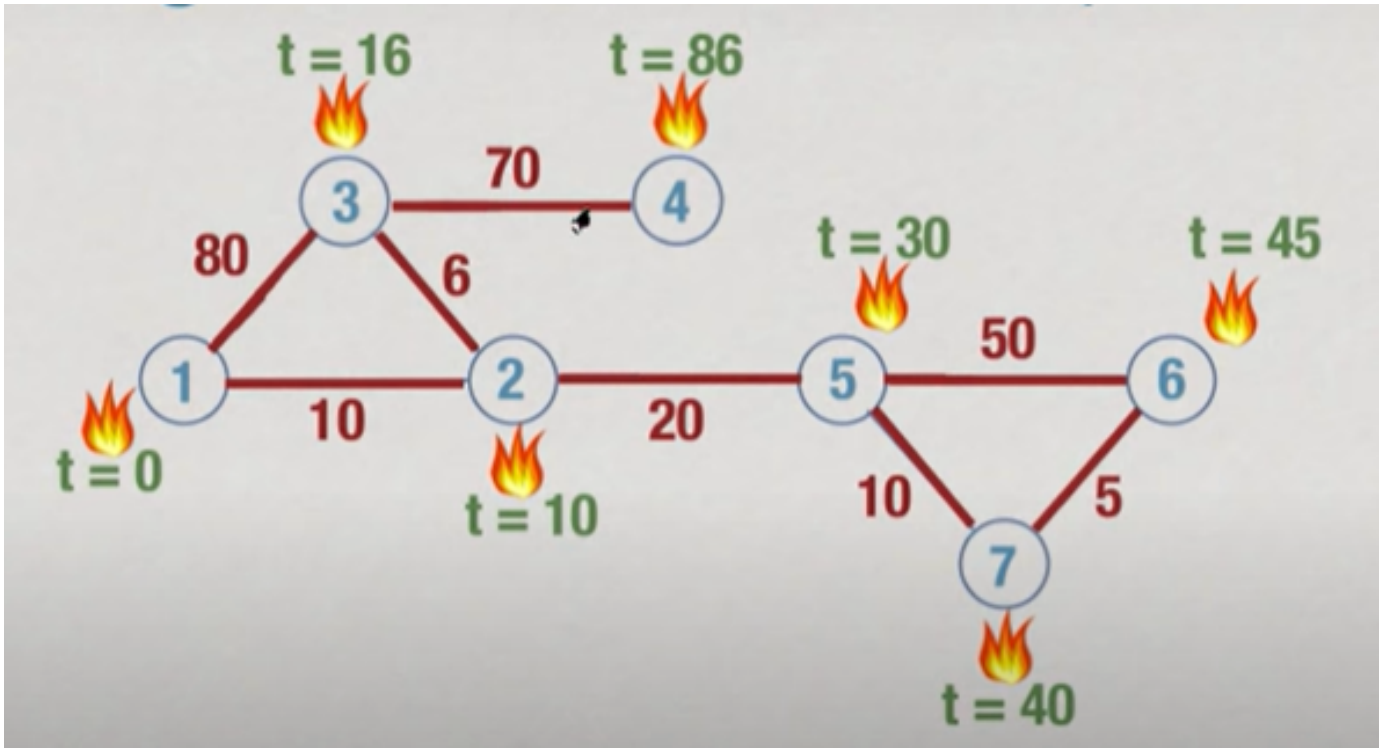
- Consider the graph[1] given below:



[1] Design and Analysis of Algorithms (NPTEL), Instructor: Prof. Madhavan Mukund, Chennai Mathematical Institute

Algorithm idea

- The state of graph[2] after all the vertices are burned:



[2]Design and Analysis of Algorithms (NPTEL), Instructor: Prof. Madhavan Mukund, Chennai Mathematical Institute

Algorithm [3]

```
DIJKSTRA(G, w, s)
INITIALIZE-SINGLE-SOURCE(G, s)
S = ∅
Q = G.V
while Q ≠ ∅
    u = EXTRACT-MIN(Q)
    S = S ∪ {u}
    for each vertex v ∈ G.Adj[u]
        RELAX(u, v, w)
```

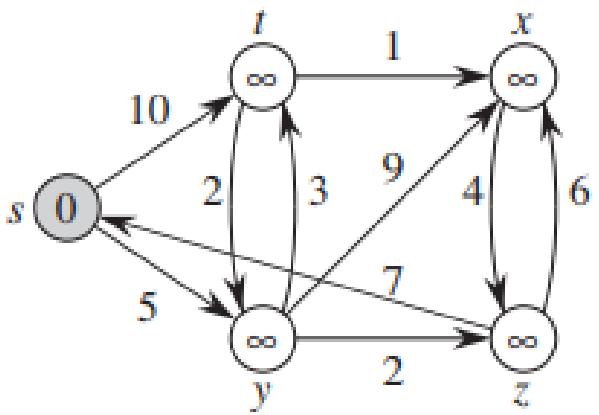
```
RELAX(u, v, w)
if v.d > u.d + w(u, v)
    v.d = u.d + w(u, v)
    v.π = u
```

[3] Introduction to Algorithms by Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein

Example [4]

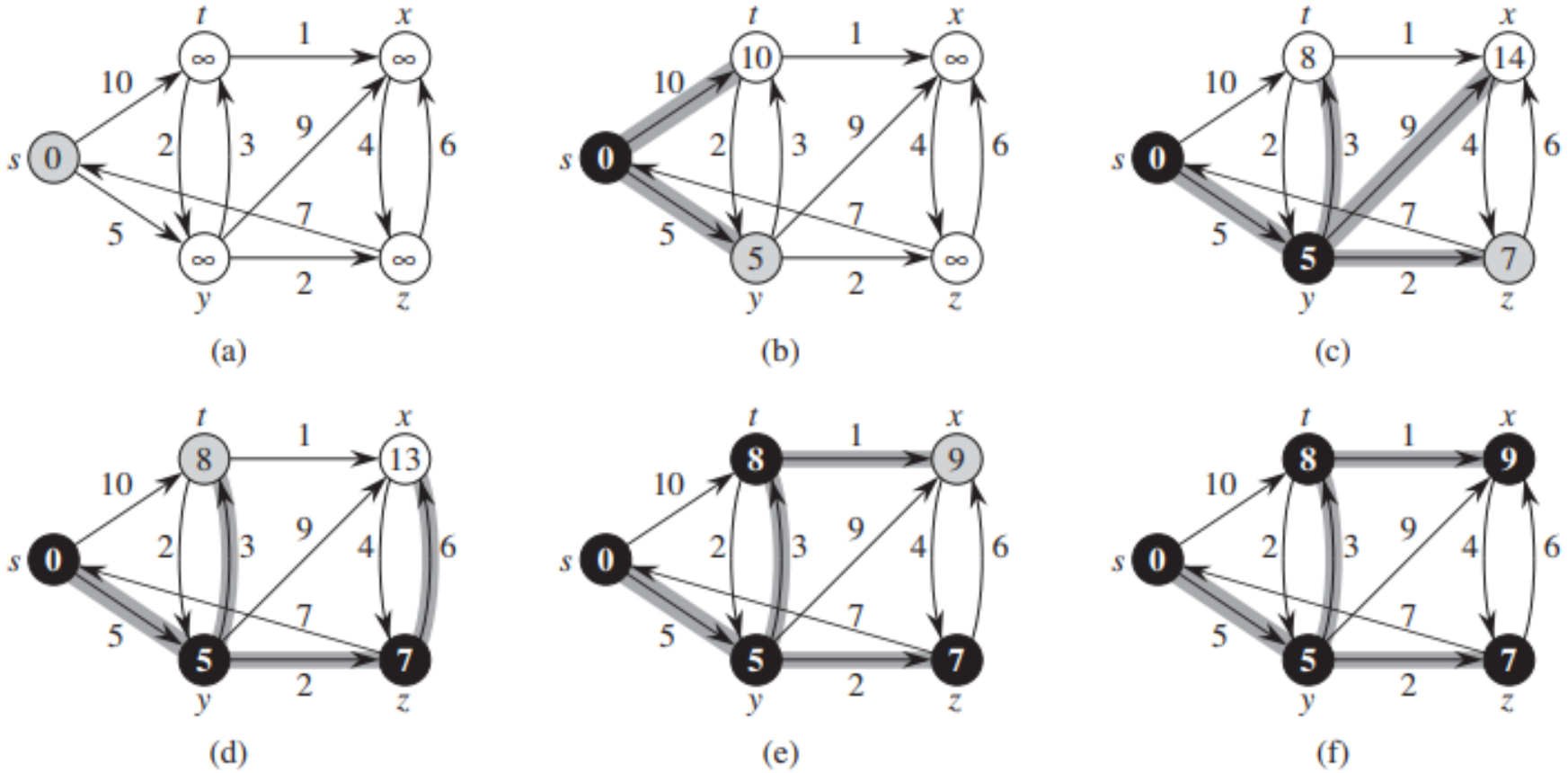
```
DIJKSTRA(G, w, s)
INITIALIZE-SINGLE-SOURCE(G, s)
S = ∅
Q = G.V
while Q ≠ ∅
    u = EXTRACT-MIN(Q)
    S = S ∪ {u}
    for each vertex v ∈ G.Adj[u]
        RELAX(u, v, w)
```

```
RELAX(u, v, w)
if v.d > u.d + w(u, v)
    v.d = u.d + w(u, v)
    v.π = u
```



[4] Introduction to Algorithms by Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein

Execution of Dijkstra's algorithm [5]



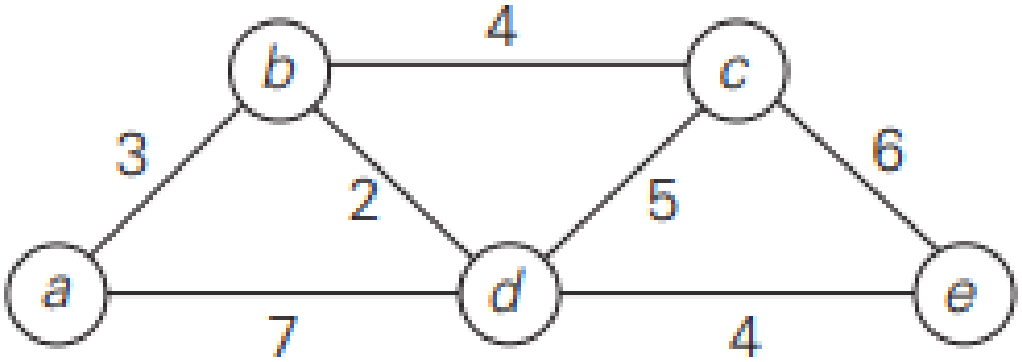
[5] Introduction to Algorithms by Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein

Algorithm idea

- The algorithm maintains a set S of vertices whose final shortest-path weights from the source s have already been determined.
- It repeatedly selects the vertex $u \in V - S$ with the minimum shortest-path estimate, adds u to S, and updates all edges leaving u.
- It uses a min-priority queue Q of vertices, keyed by their distance values from the source.
- The time efficiency of Dijkstra's algorithm depends on the data structures used for implementing the priority queue and for representing an input graph itself.
  - *while* loop takes $O(|V|)$
  - If the vertices are numbered from 1 to $|V|$, then the d value can be stored in the vth entry of the array.
  - EXTRACT-MIN then takes $O(|V|)$ time to find the minimum element, resulting in $O(|V|^2)$.

Exercise

- Compute the shortest path to all vertices from vertex *a* for the given graph using Dijkstra's algorithm:



Summary

- Single-source shortest path problem is an optimization problem that finds the minimum distance from a source to every other vertex.
- It uses greedy strategy to solve the problem.
- It uses a min-priority queue to implement the greedy strategy.
- The time depends on the data structures used for implementing the input graph and the priority queue.
- Using a Fibonacci heap for priority queue results in $O(|E| + |V| \log |V|)$.